Rodstar PayOps: Master Build Prompt (v2)

Rodstar Tech Devs. Paste this whole document into your coding agent (Claude Code or similar) at the root of an empty or existing repository.

0. How to read this prompt

You are the lead engineer building **Rodstar PayOps**. This prompt has three layers, in priority order:

1. **Principles** (section 2): never violated, even if a later instruction seems to conflict.
2. **Domain rules** (section 4): the business logic that must be correct.
3. **Phases** (section 7): the order of work and the evidence required to move on.

When something is unclear, choose the safest option for financial correctness, record the decision in `docs/DECISIONS.md`, and continue. Stop and ask only for the items listed in section 9.

1. Product

Rodstar PayOps is a multi-tenant SaaS that helps Kenyan businesses collect payments, reconcile them against invoices and bank statements, keep a correct double-entry ledger, and prepare tax invoices for KRA eTIMS.

- **Tagline:** Intelligent Payment Operations & Tax Automation
- **Primary market:** Kenya (KES, M-PESA, KRA PIN, VAT). Architecture must allow other countries and currencies later.
- **Users:** business owners, finance administrators, accountants, cashiers, auditors, read-only users, plus a platform super administrator.
- **Standard:** commercial product, not a demo. No mock data in functional paths, no dead buttons, no simulated payments, reconciliations or KRA acceptance.

2. Non-negotiable principles

1. **The backend is the single source of financial truth.** The frontend and n8n only call authenticated backend APIs. n8n never edits balances, posts ledger entries or bypasses authorization.

2. **Money is exact.** Use `Decimal` in Python and `NUMERIC(18,2)` (or integer minor units, pick one and document it) in PostgreSQL. Never use floats. Every amount carries a currency.
3. **Financial history is append-only.** Posted journal entries, payments and allocations are never edited or deleted. Corrections are reversing entries or new records linked to the original.
4. **Every financial write is atomic and idempotent.** One database transaction per business operation, protected by idempotency keys and unique constraints (provider reference, callback ID, import row hash).
5. **Tenant isolation is enforced in the backend.** Every tenant-owned table has `tenant_id`; every query goes through a tenant-scoped data layer that cannot be bypassed by a route author. Cross-tenant tests are mandatory.
6. **Never trust inbound data.** An HTTP callback is a claim, not proof of payment. Validate result codes, amounts, shortcodes, account references and duplicates before recording anything as successful.
7. **Never fabricate external behaviour.** Do not invent Daraja, bank or KRA endpoints, payloads, encryption schemes or response formats. Use official documentation provided to you. If it is missing, build the adapter interface, a clearly labelled `NotConfigured` implementation, and write exact instructions for what is needed.
8. **Honest status reporting.** A feature is "done" only when its code exists, its tests pass, and you have shown the output. A live integration is "verified" only after a successful call with real sandbox or production credentials. Otherwise report it as "implemented, unverified against live provider".
9. **Compliance claims are earned.** Never describe PayOps as KRA-certified or eTIMS-approved in code, UI, docs or marketing text until Rodstar confirms the certification in writing. An invoice is `ACCEPTED` only after an authentic success response from the approved integration.
10. **Secrets never leave the server.** No credentials in the repo, frontend bundle, logs or error messages. Provider credentials are encrypted at rest. Never store or log M-PESA PINs or full card data.

3. Scope: two releases

Build Release 1 completely before touching Release 2. This keeps the product shippable.

Release 1 (shippable core)

- Auth: password hashing (Argon2id), JWT access and refresh tokens with rotation and revocation, RBAC, TOTP-based MFA.
- Tenancy: tenant onboarding, business profile (registered name, trading name, KRA PIN, address, contacts), branches, user invites, roles and granular permissions.
- Customers, products and services, tax configuration (VAT standard, zero-rated, exempt).
- Invoices: draft, issue, numbering, PDF, credit notes, internal review workflow.
- Payments: M-PESA Daraja (OAuth, STK Push, STK callback, C2B register/validate/confirm, status query, reversal handling where supported), CSV bank statement import, manual payment recording.
- Reconciliation: rules engine, auto-match, partial and multiple allocations, overpayment, duplicates, exception queue, daily run, downloadable report.
- Ledger: chart of accounts, double-entry posting service, reversals, refunds, period close.
- eTIMS **manual review mode** plus the provider interface (no live KRA calls).
- Dashboard with all 15 navigation sections functional.
- n8n workflows WF01 to WF08 and WF10 to WF12 (see section 6).
- Audit log, notifications (email and SMS via pluggable providers), security hardening, backups, tests, docs.

Release 2 (after Release 1 is green)

- Live OSCU/VSCU adapter against official KRA documentation and credentials (WF08 and WF09 go live).
- Direct bank API or feed connectors, one bank at a time, only with documentation and credentials.
- WF09, WF13, WF14 and additional automations.
- Multi-currency, accounts payable depth, advanced reporting.

4. Domain rules

4.1 Payment lifecycle

`INITIATED → PENDING → SUCCEEDED | FAILED | CANCELLED | TIMED_OUT`, then optionally `PARTIALLY_REFUNDED | REFUNDED | REVERSED`. Each transition is recorded as an event. Every payment has a unique `(tenant_id, provider, provider_reference)`.

Callback handling order: verify authenticity and source → persist raw event in `webhook_events` → check idempotency → validate result code and amount → create or update payment → enqueue reconciliation. Always return the response the provider expects, even for duplicates.

4.2 Reconciliation matching priority

1. Exact transaction reference.
2. Exact invoice reference.
3. Exact amount and customer.
4. Exact amount and date within a configurable window.
5. Anything else goes to manual review.

Hard rules: never auto-allocate an ambiguous match (more than one candidate at the same rank); partial payments allocate and leave the remainder visible; overpayments become customer credit, not silent revenue; a reversed payment re-opens its allocations through reversing entries. Matching rules are configurable per tenant, and every match records which rule produced it and a confidence label.

4.3 Ledger

Double-entry only. A journal entry must balance (debits equal credits, per currency) before it can be posted. Posted entries are immutable. Corrections post a reversing entry and a new entry. Period closing locks postings for that period; later adjustments need an authorized role and an audit record. Minimum journals: invoice issue, payment receipt, allocation, refund, reversal, credit note, tax adjustment. Include accounts for receivables, payables, cash, bank, payment clearing, tax payable, revenue, expense.

4.4 Invoice and eTIMS states

Invoice states: `DRAFT → ISSUED_INTERNAL → PENDING_SUBMISSION → UNDER_REVIEW → SUBMITTED → ACCEPTED | REJECTED | FAILED → CORRECTED`. Only the eTIMS provider adapter may move an invoice to `ACCEPTED` or `REJECTED`, and only from a real provider response which is stored verbatim. Manual mode lets a finance user mark an invoice as submitted externally, with who, when and a reference, and it is labelled "manually recorded" everywhere it appears, never "accepted by KRA".

4.5 Bank imports

Validate file structure, headers, amounts, currency and dates. Preserve every original row. Hash rows to detect duplicates across imports. Reject malformed rows safely with a per-row error report. Produce an import summary (rows read, accepted, duplicate, rejected).

5. Architecture

5.1 Stack (pin and lock all versions)

- **Frontend:** React, TypeScript, Tailwind CSS, shadcn/ui, React Router, TanStack Query, React Hook Form, Zod, Recharts.
- **Backend:** Python, FastAPI, SQLAlchemy 2, Pydantic, Alembic, JWT, RBAC.
- **Infra:** PostgreSQL, Redis, Celery, n8n, Docker, Docker Compose, Nginx.
- **Tests:** Pytest, HTTPX, Playwright.

5.2 Boundaries

- **Celery** runs internal jobs (callback processing, reconciliation runs, report generation, retries). **n8n** orchestrates cross-system workflows (notifications, scheduling, delivery) by calling the backend with a **service account holding narrowly scoped API keys**. n8n holds no database access.
- All posting to the ledger goes through one `PostingService`. No other module writes journal tables.
- All provider code lives in `integrations/<provider>/` behind an interface (`PaymentProvider`, `BankFeedProvider`, `TaxAuthorityProvider`). Business logic never imports provider specifics.
- Tenant scoping lives in a shared data-access layer and FastAPI dependency; routes cannot query tenant tables without it.

5.3 Repository layout

```
rodstar-payops/

backend/app/{api,core,models,schemas,services,integrations/{daraja,banks,etim
s},accounting,reconciliation,workers}
  backend/{migrations,tests}
  frontend/src/{components,pages,features,services,hooks,layouts}
  n8n/workflows/
  infrastructure/{nginx,scripts}
  docs/            # DECISIONS.md, API.md, SECURITY.md, DEPLOY.md,
TROUBLESHOOTING.md
  docker-compose.yml  docker-compose.sandbox.yml  .env.example  README.md
```

5.4 API

Versioned under `/api/v1`. Modules: auth, tenants, users, branches, customers, products, invoices, payments, payment-providers, bank-statements, reconciliation, ledger, tax, etims, reports, notifications, audit, admin. Every endpoint has request and response schemas, pagination, filtering, sorting, rate limiting, consistent error format and generated OpenAPI docs.

5.5 Data model (minimum)

tenants, users, roles, permissions, branches, customers, products, invoices, invoice_items, credit_notes, payments, payment_allocations, payment_providers, bank_accounts, bank_transactions, reconciliation_runs, reconciliation_matches, reconciliation_exceptions, chart_of_accounts, journal_entries, ledger_entries, tax_configurations, tax_submissions, notifications, automation_jobs, audit_logs, webhook_events, idempotency_records, refresh_tokens.

Foreign keys, indexes, tenant-scoped unique constraints, and complete Alembic migrations that run cleanly on an empty database and downgrade where practical.

6. Automations (n8n)

Deliver importable JSON for each workflow, with retries, error branches, execution logging and an alert on final failure. Every workflow authenticates to the backend and states which endpoints it calls.

Group	Workflows
Payments	WF01 M-PESA intake, WF02 STK Push initiation, WF03 callback processing
Reconciliation	WF04 invoice reconciliation, WF05 unmatched exceptions, WF06 bank import, WF07 daily bank reconciliation, WF14 period closing
Tax	WF08 eTIMS submission queue, WF09 failed submission recovery
Customer comms	WF10 invoice delivery, WF11 payment reminders
Reporting and approvals	WF12 daily financial report, WF13 refund approval notifications

WF08 and WF09 ship in manual-mode form in Release 1 and go live with the approved KRA adapter in Release 2.

7. Working protocol and phases

7.1 Rules of engagement

- Work directly in the repository. Create real files. Do not paste large code dumps into chat.
- First action: inspect the repository, then write a concise plan to `docs/PLAN.md`.
- Complete one phase at a time. Do not start the next phase until the current phase's exit criteria pass.
- Run builds, migrations and tests yourself and fix failures. Do not leave known failures as notes.
- Commit at the end of each phase with a clear message.
- Keep a running `docs/DECISIONS.md` for every non-obvious choice and every assumption.

7.2 Phases and exit criteria

Phase	Build	Exit criteria (all must be shown passing)
1. Foundation	Repo structure, Docker Compose, Postgres, Redis, config and secrets handling, auth, MFA, tenancy, RBAC, base migrations, audit logging, health checks	Stack starts with one command; migrations apply to an empty DB; register-login-refresh-revoke flow works; cross-tenant access test fails closed
2. Payments	Daraja OAuth, STK Push, STK and C2B callbacks, payment records, idempotency, webhook event store, CSV bank import	Duplicate callback creates one payment; failed and timed-out payments handled; malformed CSV rows rejected safely; sandbox call succeeds if credentials supplied, else marked unverified
3. Reconciliation	Rules engine, allocations, partials, overpayments, duplicates, exception queue, daily run, reports	Fixture-based accuracy tests for every rule; ambiguous matches never auto-allocate; exceptions resolvable with audit trail
4. Accounting	Chart of accounts, PostingService, journals, reversals, refunds, credit notes, period close, customer balances	Property tests: every posted entry balances; posted entries immutable; reversal preserves history
5. Invoicing and eTIMS	Invoices, VAT, numbering, PDF, credit notes, manual review mode, `TaxAuthorityProvider` interface and `NotConfigured` adapter, submission queue and retries	Manual mode never produces `ACCEPTED`; swapping in a fake adapter in tests requires no change to invoice code
6. Automation	n8n workflows, service account and scoped keys, alerting	Each workflow imports and runs against a running backend; failures retry then alert

Phase	Build	Exit criteria (all must be shown passing)
7. Frontend	All 15 sections wired to real APIs, permission-aware UI, loading, empty and error states, mobile and desktop	Playwright journeys for the main flows pass; no unwired controls
8. Production readiness	Security tests, backup and restore drill, load sanity check, docs, deployment guide	Full sandbox journey (invoice → STK Push → callback → reconcile → ledger → report) passes end to end

7.3 Report at the end of each phase

1. Files created or modified.
2. Features implemented.
3. Tests run, with actual output summary.
4. What is verified vs implemented-but-unverified.
5. Blockers and what is needed from Rodstar.
6. Exact commands to run the application.

8. Quality bars

Security

Argon2id hashing, MFA, refresh rotation and reuse detection, RBAC with granular permissions, rate limiting, input validation, parameterised queries only, CSRF protection where cookies are used, XSS-safe rendering, security headers, HTTPS enforcement, webhook authenticity checks, encrypted provider credentials with key rotation notes, audit logging of sensitive actions, dependency vulnerability scanning in CI, tested backups and restore procedure. Add dedicated tests for cross-tenant reads, writes and ID guessing on every module.

Frontend

A polished fintech feel, not a generic template: restrained palette, clear typography, consistent spacing, strong information hierarchy, meaningful status badges, accessible forms, useful empty states, professional charts. Use the Rodstar brand palette and logo

assets if provided; otherwise ask once (see section 9). Sections: Overview, Payments, Transactions, Reconciliation, Invoices, eTIMS, Customers, Products and Services, Banking, Accounting, Reports, Automations, Team and Permissions, Business Settings, Audit Logs. Overview shows today's and monthly collections, outstanding invoices, reconciled vs unmatched, failed payments, pending and accepted eTIMS, revenue trend and payment-method breakdown, all from real queries.

Testing

Unit tests for money, matching and ledger logic; integration tests against a real Postgres; provider adapters tested against recorded fixtures plus failure injection; Playwright for user journeys. CI runs lint, type checks, tests and dependency scan.

9. Ask the owner before you start (one message, then proceed)

Ask these together, then continue with documented assumptions for anything unanswered:

1. Do you have Daraja sandbox credentials now, and will production use a Paybill or a Till Number?
2. Where will it be deployed first (for example Hetzner VPS) and which domain should be used?
3. Which email and SMS providers should be wired in (or should the first release use email only)?
4. Are brand colours and logo files available, or should a neutral Rodstar theme be used?
5. Do you have KRA eTIMS onboarding documentation or sandbox access, or should eTIMS stay in manual mode for now?

10. Definition of done for Release 1

- [ ] A business can register, configure its tenant, invite staff and assign permissions.
- [ ] Customers and invoices can be created and issued.
- [ ] STK Push can be sent with configured credentials, and a valid callback records exactly one payment.
- [ ] Duplicate callbacks and re-imported bank rows create no duplicates.
- [ ] Partial payments update invoice balances correctly; overpayments become customer credit.

- [ ] Bank transactions reconcile; unmatched items land in the exception queue and can be resolved with an audit record.
- [ ] Every posted journal entry balances; refunds and reversals preserve history.
- [ ] eTIMS manual mode works and never shows false KRA acceptance; the provider interface accepts a real adapter without rewriting invoicing.
- [ ] n8n workflows run against authenticated backend APIs only.
- [ ] Unauthorized users and other tenants cannot reach protected data.
- [ ] Main journeys work on mobile and desktop.
- [ ] Docs complete: README, API docs, deployment guide, security checklist, troubleshooting guide, `.env.example`, separate sandbox config.

Begin now: inspect the repository, ask the section 9 questions in one message, write `docs/PLAN.md`, then complete Phase 1.